Dog Breed Identification Using Transfer Learning

Project Description:

One of the most important and rapidly growing areas in Artificial Intelligence is Computer Vision, which enables machines to interpret and understand visual data. Image classification plays a vital role in many real-world applications such as healthcare, security, agriculture, and animal identification. Identifying dog breeds manually requires expert knowledge and can be time-consuming. Therefore, developing an automated dog breed prediction system becomes highly important.

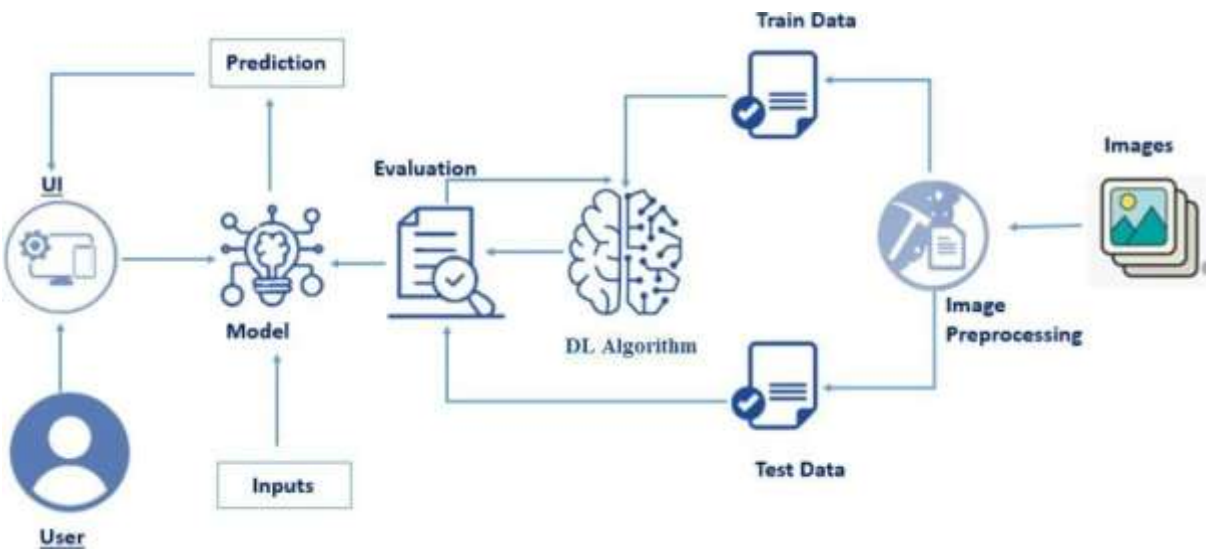
The prediction of dog breeds from images is a challenging task due to variations in lighting, background, pose, fur texture, and similarity between certain breeds. However, by applying Deep Learning techniques, especially Convolutional Neural Networks (CNN), it is possible to accurately classify dog breeds from images. This helps in applications such as veterinary services, pet adoption centers, animal shelters, and educational platforms.

In this project, we use a dataset obtained from Kaggle, which contains labeled images of multiple dog breeds. From the complete dataset, 20 dog breeds were selected and organized into separate directories for training purposes. Image preprocessing techniques such as resizing, normalization, and data augmentation were applied to improve model performance and reduce overfitting.

We implemented Transfer Learning using the pre-trained deep learning model VGG16. The model was fine-tuned by unfreezing selected layers and adding fully connected layers for classification. The system was trained using the Adam optimizer and categorical cross-entropy loss function. After training, the best-performing model was saved in H5 format (dogbreed.h5).

Finally, the trained model was integrated with Flask to build a web-based application. The user can upload a dog image, and the system predicts the breed in real-time. This project demonstrates the practical implementation of deep learning techniques combined with web deployment for real-world applications.

Technical Architecture:



Pre requisites:

To complete this project, you must require the following software's, concepts, and packages

Anaconda Navigator is a free and open-source distribution of the Python and R programming languages for data science and machine learning related applications. It can be installed on Windows, Linux, and macOS. Conda is an open-source, cross-platform, package management system. Anaconda comes with so very nice tools like JupyterLab, Jupyter Notebook,

QtConsole, Spyder, Glueviz, Orange, Rstudio, Visual Studio Code. For this project, we will be using Jupyter notebook and VS code.

To install Anaconda navigator and to know how to use Jupyter Notebook & Spyder using Anaconda watch the video
Link: Click here to watch the video

1. **To build Machine learning models you must require the following packages**

   - **Numpy:** It is an open-source numerical Python library. It contains a multidimensional array and matrix data structures and can be used to perform mathematical operations
   - **Scikit-learn:** It is a free machine learning library for Python. It features various algorithms like support vector machine, random forests, and k-neighbors, and it also supports Python numerical and scientific libraries like NumPy and SciPy
   - **Flask:** Web framework used for building Web applications

- **Python packages:**

    - open anaconda prompt as administrator
    - Type “pip install numpy” and click enter.
    - Type “pip install pandas” and click enter.
    - Type “pip install scikit-learn” and click enter.
    - Type “pip install tensorflow==2.12.0” and click enter.
    - Type “pip install keras==2.12.0” and click enter.
    - Type “pip install Flask” and click enter.

Deep Learning Concepts

- **CNN:** a convolutional neural network is a class of deep neural networks, most commonly applied to analyzing visual imagery.
  CNN Basic

- **VGG19**: VGG19 is a deep convolutional neural network architecture for image classification, consisting of 19 layers with small convolution filters.
  VGG19

- **Flask:** Flask is a popular Python web framework, meaning it is a third-party Python library used for developing web applications.
  Flask Basics

Prior Knowledge:

You must have prior knowledge of following topics to complete this project.

- **ML Concepts**
    - Supervised learning: https://www.javatpoint.com/supervised-machine-learning
    - Unsupervised learning: https://www.javatpoint.com/unsupervised-machine-learning
    - Regression and classification
    - Decision tree: https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm
    - Random forest: https://www.javatpoint.com/machine-learning-random-forest-algorithm
    - KNN: https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning
    - Xgboost: https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/
    - Evaluation metrics: https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/
- **Flask Basics** : https://www.youtube.com/watch?v=lj4I_CvBnt0

Project Objectives:

By the end of this project you will:

- Know fundamental concepts and techniques of Convolutional Neural Network.
- Gain a broad understanding about data.
- Know how to pre-process/clean the data using different data preprocessingtechniques.
- know how to build a web application using the Flask framework.

Project Flow:

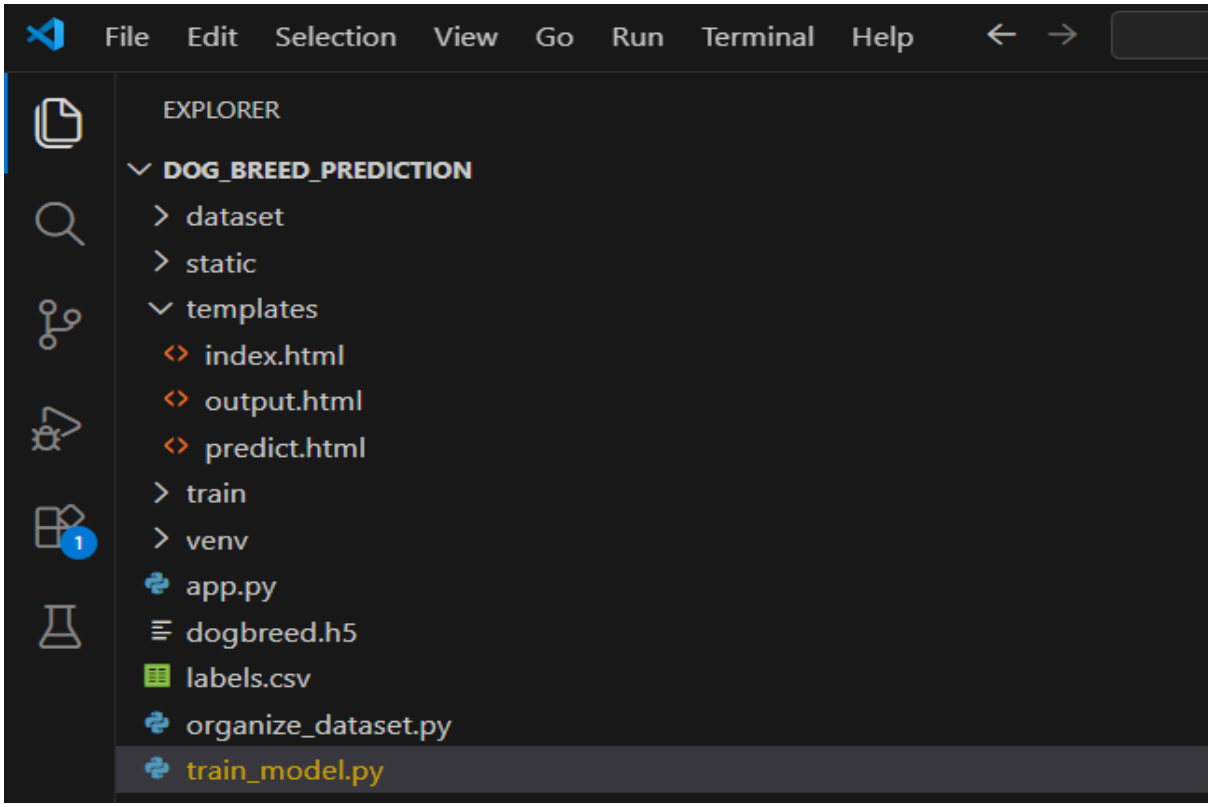
- The user interacts with the UI (User Interface) to choose the image.
- The chosen image is analyzed by the model which is integrated with flask application.
- CNN Models analyze the image, then prediction is showcased on the Flask UI.

To accomplish this, we have to complete all the activities listed below,

- Data collection
  - Create Train and Test Folders.
- Data pre-processing
  - Import the ImageDataGenerator library
  - Configure ImageDataGenerator class
  - ApplyImageDataGenerator functionality to Train dataset and Test dataset
- Model building
  - Import the model building libraries
  - Initializing the VGG19.
  - Initializing the model
  - Adding Fully connected Layer
  - Configure the Learning Process
  - Training and Testing the model.Save the Model
- Application Building
  - Create an HTML file
  - Build python code

Project Structure:

Create the Project folder which contains files as shown below



- We are building a Flask Application that needs HTML pages stored in the templates.folder and a python script app.py for server-side scripting
- we need the model which is saved and the saved model in this content is a dogbreed.h5
- templates folder contains index.html, predict.html & output.html pages.
- The Static folder contains css file, images.

Milestone 1: Data Collection

In this milestone First, we will collect images of Dog Breeds then organized into subdirectories based on their respective names as shown in the project structure. Create folders of types of Dog Breeds that need to be recognized. In this project, we have collected images of 20 types of Images like affenpinscher, beagle, appenzeller, basset, bluetick, boxer, cairn, doberman, german_shepherd, golden_retriever, kelpie, komondor, leonberg,

mexican_hairless, pug, redbone, shih-tzu, toy_poodle, vizsla, whippet they are saved in the respective sub directories with their respective names.

Activity 1: Download the dataset

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link: https://www.kaggle.com/competitions/dog-breed-identification/data?select=train

Milestone 2: Visualizing and analysing the data

As the dataset is downloaded from **Kaggle**, let us read and understand the data properly with the help of visualization and analysis techniques.

Note: There are many techniques available to understand image datasets. Here, we have used some commonly used visualization and analysis methods. Additional techniques can also be applied.

Activity 1: Importing the libraries

Import the necessary libraries as shown in the image. (optional) Here we have used visualization style as fivethirtyeight.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import os
```

Activity 2: Read the Dataset

Our dataset format might be in .csv, excel files, .txt, .json, etc. We can read the dataset with the help of pandas.In pandas we have a function called read_csv() to read the dataset. As a parameter we have to give the directory of csv file.

```
df = pd.read_csv("labels.csv")
df.head()
```

```
Total rows in CSV: 10222
Total images copied: 10222
```

Milestone 3: Data Pre-processing

As we have understood how the data is lets pre-process the collected data.

The download data set is not suitable for training the machine learning model as it might have so much of randomness so we need to clean the dataset properly in order to fetch good results. This activity includes the following steps.

- Handling missing values
- Handling categorical data
- Handling outliers
- Scaling Techniques
- Splitting dataset into training and test set
- Images have different sizes
- Pixel values are not normalized
- Data may be imbalanced
- Labels are in categorical (string) format

Note: These are the general steps of pre-processing the data before using it for machine learning. Depending on the condition of your dataset, you may or may not have to go through all these steps.

Checking for null values

- Let's find the shape of our dataset first, To find the shape of our data, df.shape method is used. To find the data type, df.info() function is used.

```
print(labels.isnull().sum())
```

```
id        0
breed     0
dtype: int64
```

- For checking the null values, df.isnull() function is used. To sum those null values we use .sum() function to it. From the below image we found that there are no null values present in our dataset. So we can skip handling of missing values step.

```
#saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))
```

Milestone 4: Importing the Model Building Libraries

To initialize the VGG19 model, the weights are usually pre-trained on the ImageNet dataset, which is a large-scale dataset of images belonging to 1,000 different categories. These pre-trained weights can be downloaded from the internet, and they can be used as a starting point to fine-tune the model for a specific task, such as object recognition or classification.

```
import os
import numpy as np
from flask import Flask, render_template, request, url_for
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
```

```
from tensorflow.keras.applications.vgg19 import VGG19
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
```

The model will be initialized with the pre-trained weights from the ImageNet dataset, and the last fully connected layer will be excluded from the model architecture.

The loop that follows freezes the weights of all the layers in the VGG19 model by setting i.trainable=False` for each layer in the model. This is done to prevent the weights from being updated during training, as the model is already pre-trained on a large dataset.Finally, a `Flatten()` layer is added to the output of the VGG19 model to convert the output tensor into a 1D tensor.The resulting model can be used as a feature extractor for transfer learning or as a starting point for building a new model on top of it.

Milestone 5: Application Building

In this section, we will be building a web application that is integrated to the model we built. A UI is provided for the uses where he has to enter the values for predictions. The enter values are given to the saved model and prediction is showcased on the UI.

This section has the following tasks

- Building HTML Pages
- Building serverside script

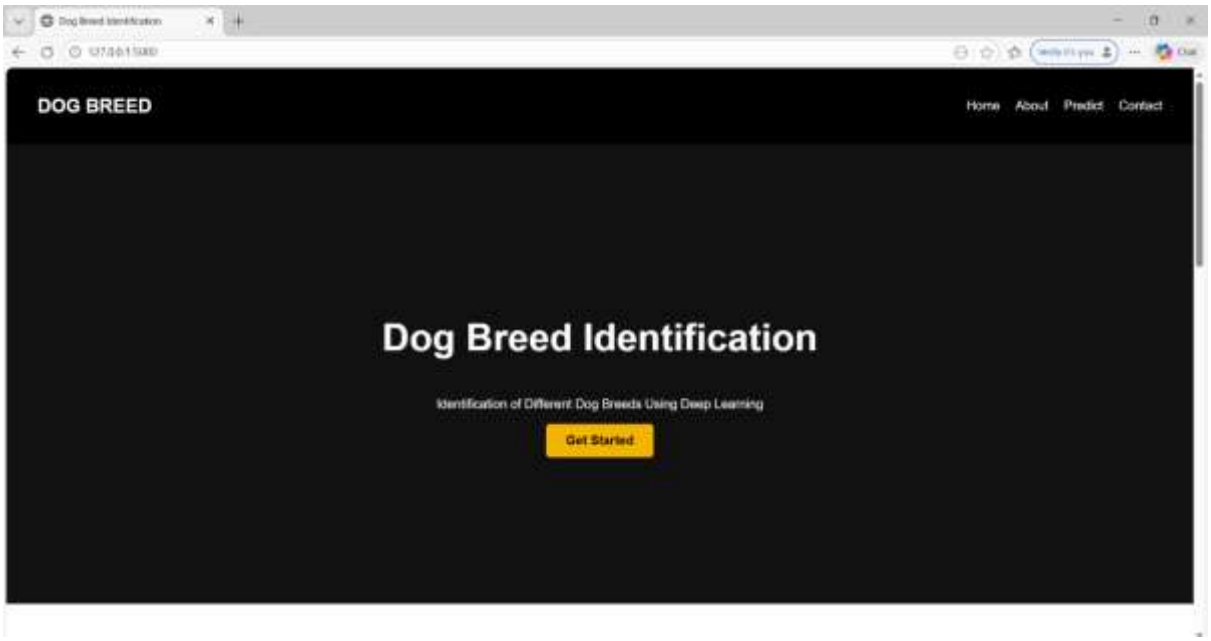
Activity1: Building Html Pages:

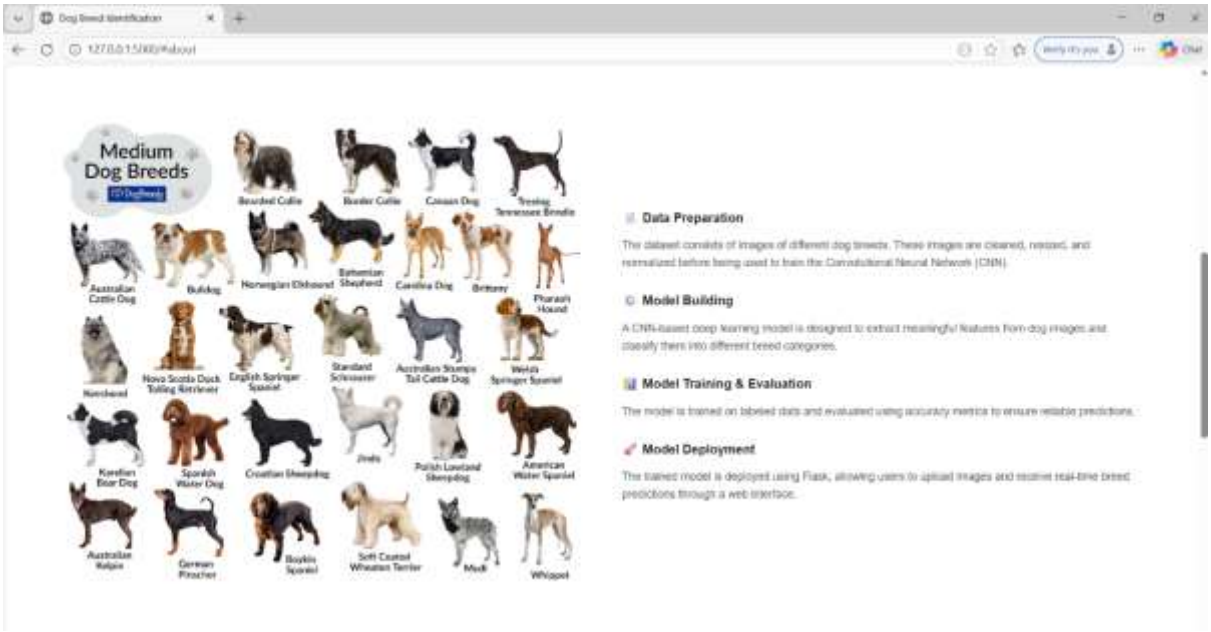
For this project create three HTML files namely

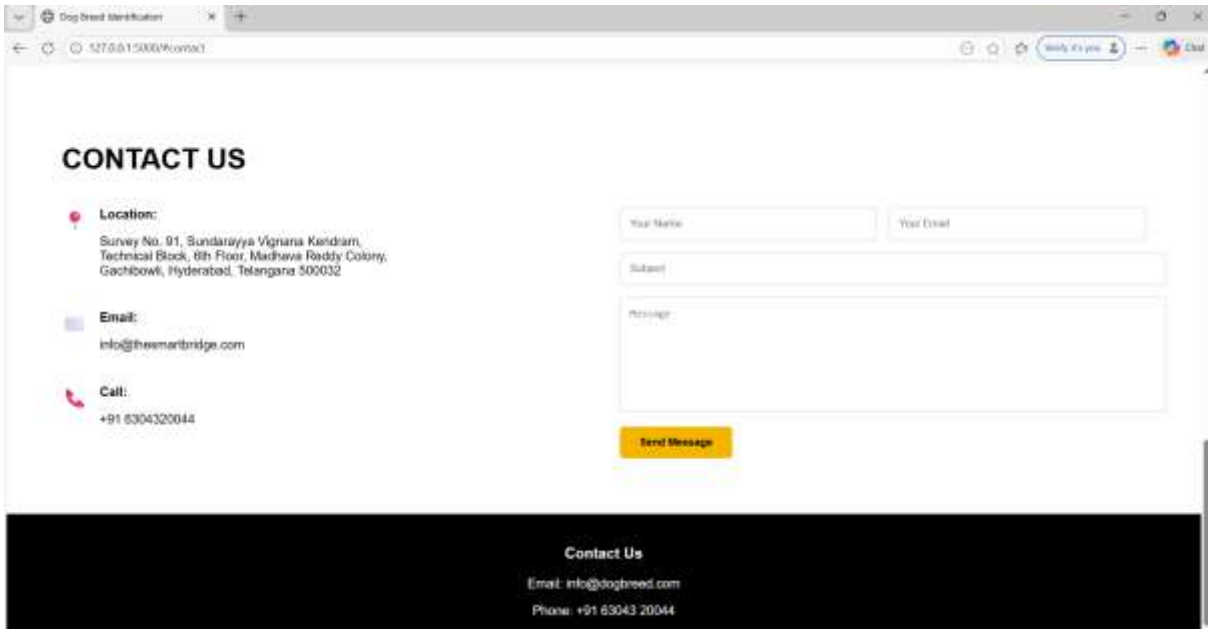
- home.html
- predict.html
- output.html

and save them in templates folder.

Let's see how our home.html page looks like:

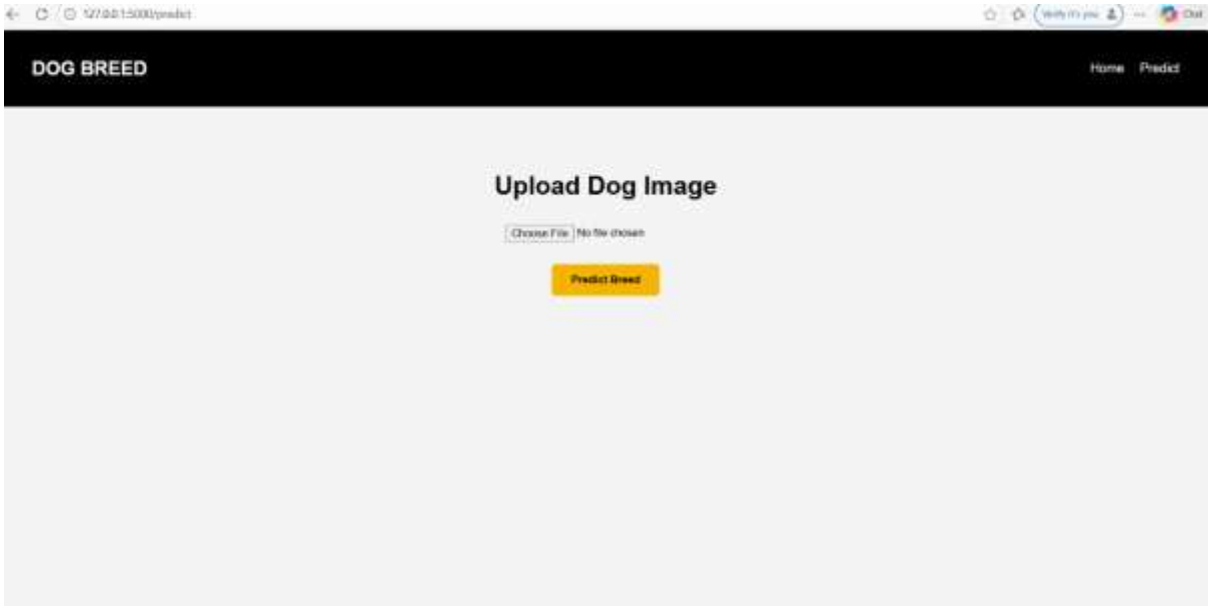






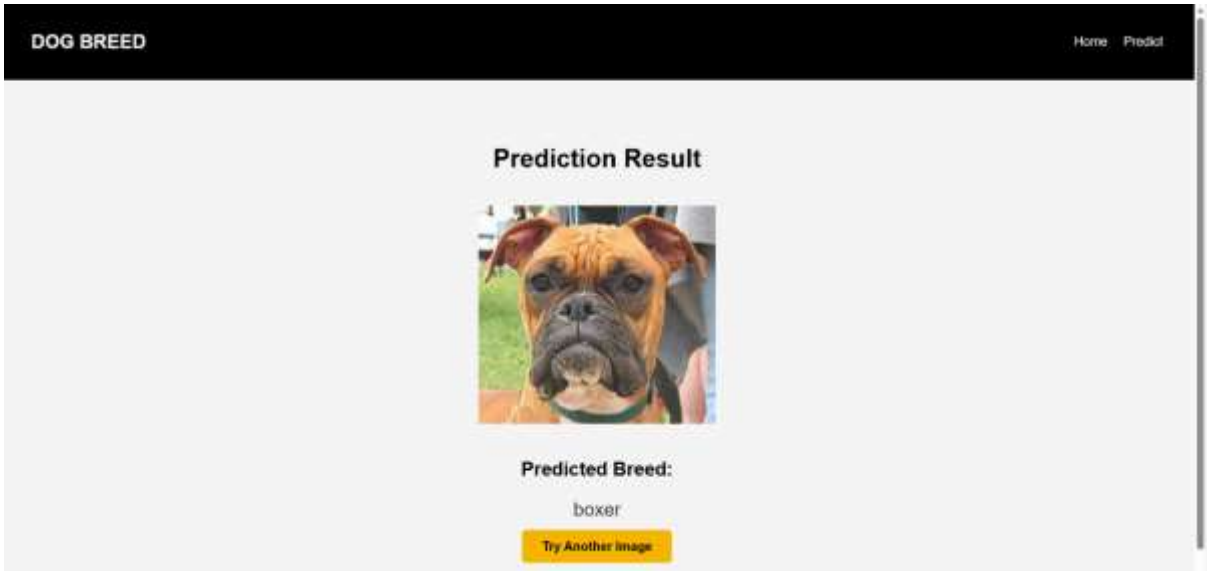
Now when you click on predict button from top right corner you will get redirected to predict.html

Lets look how our predict.html file looks like:



Now when you click on submit button from left bottom corner you will get redirected to submit.html

Lets look how our submit.html file looks like:



Activity 2: Build Python code:

Import the libraries

```
import os
import numpy as np
from flask import Flask, render_template, request, url_for
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
```

Load the saved model. Importing flask module in the project is mandatory. An object of Flask class is our WSGI application. Flask constructor takes the name of the current module (__name__) as argument.

```
app = Flask(__name__)

# Load trained model
model = load_model("dogbreed.h5")
```

Render HTML page:

```
# ---------------- HOME PAGE ----------------
@app.route("/")
def home():
    return render_template("index.html")
```

Here we will be using declared constructor to route to the HTML page which we have created earlier.

In the above example, '/' URL is bound with home.html function. Hence, when the home page of the web server is opened in browser, the html page will be rendered. Whenever you enter the values from the html page the values can be retrieved using POST Method.

Retrieves the value from UI:

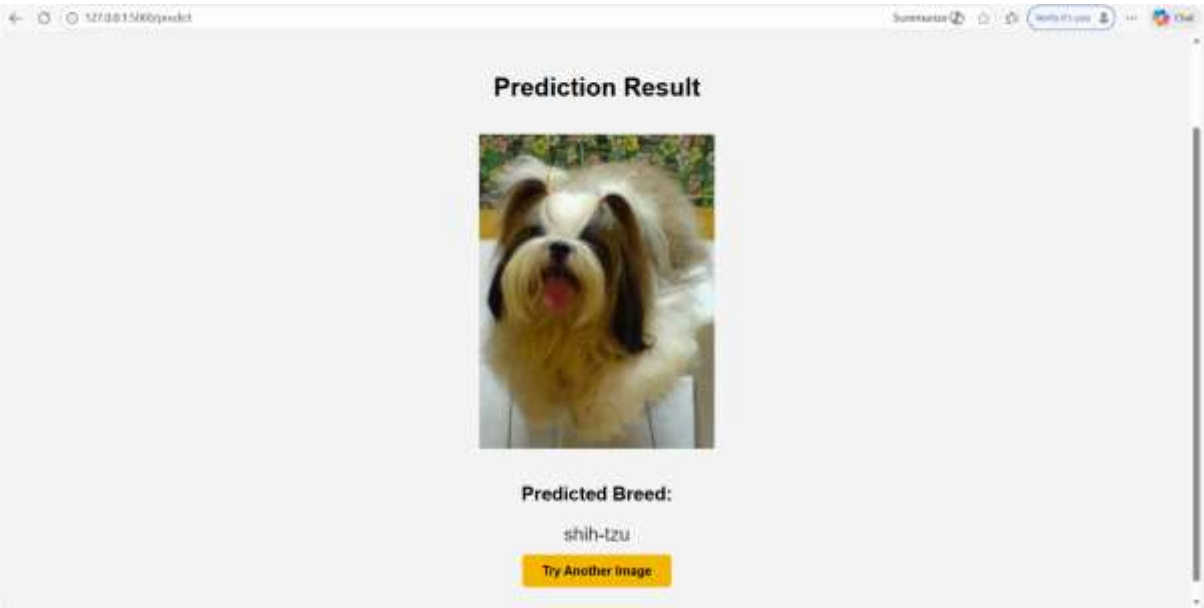
```python
# ---------------- PREDICT PAGE ----------------
@app.route("/predict", methods=["GET", "POST"])
def predict():

    if request.method == "POST":

        if "file" not in request.files:
            return "No file uploaded"

        file = request.files["file"]

        if file.filename == "":
            return "No selected file"

        # Save image
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
        file.save(filepath)

        # Preprocess image
        img = image.load_img(filepath, target_size=(224, 224))
        img_array = image.img_to_array(img)
        img_array = img_array / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        # Predict
        predictions = model.predict(img_array)
        predicted_class = class_names[np.argmax(predictions)]

        image_url = url_for('static', filename='uploads/' + file.filename)

        return render_template(
            "output.html",
            prediction=predicted_class,
            image_path=image_url
        )

    return render_template("predict.html")
```

Here we are routing our app to predict() function. This function retrieves all the values from the HTML page using Post request. That is stored in an array. This array is passed to the model.predict() function. This function returns the prediction. And this prediction value will rendered to the text that we have mentioned in the submit.html page earlier.

Main Function:

```python
if __name__ == "__main__":
    app.run(debug=True)
```

Activity 3: Run the application

- Open VS code from the start menu
- Navigate to the folder where your python script is.
- Now type “python app.py” command
- Navigate to the localhost where you can view your web page.
- Click on the predict button from top left corner, enter the inputs, click on the submit button, and see the result/prediction on the web.

```
base) D:\TheSmartBridge\Projects\2. DrugClassification\Drug c
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a p
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```